Verification and Validation Report: Software Engineering

Team #12, Streamliners
Mahad Ahmed
Abyan Jaigirdar
Prerna Prabhu
Farhan Rahman
Ali Zia

March 10, 2026

1 Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

2 Symbols, Abbreviations and Acronyms

Symbol	Description
LEMS	Large Event Management System
SRS	Software Requirements Specification
VnV	Verification and Validation
FR1–FR5	Functional Requirement categories
FRx-Tyy	Functional Requirement test case with ID FR*x*-T*yy*
RBAC	Role-Based Access Control
TPPS	Ticket Purchase and Payment System
BTR	Bus and Table Registration
WPS	Waiver and Preference Submission (legacy module, no longer implemented)
ECM	Event Creation and Management
TP#	Test Procedure identifier in nonfunctional test tables (e.g., TP1, TP2)
Jest	JavaScript testing framework used for automated tests
Stripe	Third-party payment processing service used for ticket payments and refunds

Contents

1 Revision History **i**

2 Symbols, Abbreviations and Acronyms **ii**

3 Functional Requirements Evaluation **1**
- 3.1 Ticket Reservation and Lockout (FR1) . . . 1
- 3.2 Payment Processing (FR2) . . . 3
- 3.3 Refund Request Management (FR3) . . . 5
- 3.4 User Management (FR4) . . . 6
- 3.5 Role-Based Access Control (RBAC) (FR5) . . . 7

4 Nonfunctional Requirements Evaluation **8**
- 4.1 Performance (NFR1) . . . 8
- 4.2 Usability (NFR2) . . . 10
- 4.3 Security Compliance (NFR3) . . . 11

5 Comparison to Existing Implementation **12**

6 Unit Testing **12**
- 6.1 Event Creation and Management Module . . . 12
- 6.2 Ticket Purchase and Payment System Module . . . 14
- 6.3 Bus and Table Registration Module . . . 16
- 6.4 Role-Based Access Control (RBAC) Module . . . 17
- 6.5 Event Signup Screen Module . . . 18
- 6.6 Seat Reservation Lockout Module . . . 20
- 6.7 Waivers and Preference Submission Module . . . 22

7 Changes Due to Testing **22**
- 7.1 Refund Functionality . . . 22
- 7.2 Admin Event Editing . . . 22

8 Automated Testing **22**

9 Trace to Requirements **24**

10 Trace to Modules **26**

11 Code Coverage Metrics **28**

List of Tables

1 Traceability Between Tests and Functional Requirements . . . 25
2 Traceability Between Tests and Modules 27

List of Figures

This document ...

3 Functional Requirements Evaluation

This section evaluates whether the Large Event Management System (LEMS) satisfies the functional requirements defined in the Software Requirements Specification (SRS). Functional requirements describe the expected behaviour of the system and define how users interact with the platform to perform tasks such as reserving tickets, completing event registrations, processing payments, requesting refunds, and managing user accounts.

The evaluation was performed using automated backend unit tests implemented with the Jest testing framework. These tests validate that controllers, services, and authorization guards behave correctly under both normal and edge-case conditions. The tests simulate realistic user actions and verify that the system enforces capacity constraints, maintains data integrity, and prevents unauthorized access.

The functional areas evaluated are:

- Ticket Reservation and Lockout
- Payment Processing
- Refund Request Management
- User Management
- Role-Based Access Control (RBAC)

3.1 Ticket Reservation and Lockout (FR1)

The ticket reservation subsystem ensures that event seats cannot be oversold and that users cannot reserve the same seat simultaneously. The system uses temporary reservations to lock seats during checkout while payment is being processed.

Test Case FR1-T01: Successful Seat Reservation

1. **Test ID:** FR1-T01-1

   Initial State: An event exists with available table seats and remaining capacity.

   Input: A user attempts to create a checkout session for a ticket.

   Expected Output: The system reserves an available seat, creates a reservation record, and generates a Stripe checkout session.

   Test Case Derivation: Ensures that the system correctly reserves seats before payment and prevents multiple users from selecting the same seat simultaneously.

Test Case FR1-T02: Event Capacity Enforcement

1. **Test ID:** FR1-T02-1

   Initial State: The event has reached its maximum capacity.

   Input: A user attempts to reserve a ticket.

   Expected Output: The system returns an error message indicating that the event is full.

   Test Case Derivation: Confirms that the system correctly prevents ticket purchases once the capacity limit is reached.

Test Case FR1-T03: Seat Lockout When No Seats Available

1. **Test ID:** FR1-T03-1

   Initial State: All table seats are reserved or locked by other users.

   Input: A user attempts to create a checkout session.

   Expected Output: The system returns an error indicating that no table seats remain.

   Test Case Derivation: Validates the seat lockout mechanism that prevents double booking.

Test Case FR1-T04: Reservation Completion After Payment

1. **Test ID:** FR1-T04-1

   Initial State: A reservation exists associated with a Stripe checkout session.

   Input: The payment is successfully completed.

   Expected Output: The system creates a ticket record, assigns the seat to the user, records the payment, and deletes the temporary reservation.

   Test Case Derivation: Ensures that successful payments correctly finalize ticket registration.

Test Case FR1-T05: Reservation Release

1. **Test ID:** FR1-T05-1

   Initial State: A temporary seat reservation exists for a checkout session.

   Input: The checkout session is cancelled or expires.

   Expected Output: The reservation is deleted and the seat becomes available again.

   Test Case Derivation: Confirms that seat reservations do not remain locked indefinitely.

Test Case FR1-T06: Concurrent Reservation Handling

1. **Test ID:** FR1-T06-1

   Initial State: Only one seat remains available for an event.

   Input: Two users attempt to reserve the final seat simultaneously.

   Expected Output: One reservation succeeds while the other receives an error indicating that no seats remain.

   Test Case Derivation: Verifies that the system prevents race conditions and maintains data consistency during concurrent access.

3.2 Payment Processing (FR2)

The payment subsystem manages financial transactions associated with event ticket purchases. Payments are processed through the Stripe payment service and recorded within the system.

The functional tests verify that checkout sessions are created correctly, payments are recorded after successful transactions, and reservations are finalized into tickets once payment is completed.

Test Case FR2-T01: Checkout Session Creation

1. **Test ID:** FR2-T01-1

   Initial State: An event exists with a valid Stripe price ID.

   Input: A user attempts to purchase a ticket.

   Expected Output: The system creates a Stripe checkout session and returns the session URL and session identifier.

   Test Case Derivation: Confirms that the payment service successfully initializes a Stripe checkout session for ticket purchases.

Test Case FR2-T02: Payment Recording

1. **Test ID:** FR2-T02-1

   Initial State: A Stripe payment has been completed successfully.

   Input: The system receives payment confirmation containing the Stripe payment intent ID and transaction details.

   Expected Output: The system records the payment in the database and associates it with the corresponding user, event, and ticket.

   Test Case Derivation: Ensures that successful payments are properly recorded for financial tracking and auditing.

Test Case FR2-T03: Reservation Finalization After Payment

1. **Test ID:** FR2-T03-1

   Initial State: A seat reservation exists for a user and a Stripe checkout session has been completed.

   Input: The system processes the successful payment event.

   Expected Output: The system creates a ticket, assigns the reserved seat to the user, records the payment, and deletes the temporary reservation.

   Test Case Derivation: Validates the full payment completion workflow from reservation to finalized ticket.

3.3 Refund Request Management (FR3)

The refund subsystem allows users to request refunds for previously purchased tickets and enables administrators to review and process these requests. The tests verify that refund requests can be created, retrieved, and processed correctly.

Test Case FR3-T01: Refund Request Creation

1. **Test ID:** FR3-T01-1

   Initial State: A user has purchased a ticket for an event.

   Input: The user submits a refund request containing the ticket identifier and a reason for the refund.

   Expected Output: A new refund request record is created and associated with the user and ticket.

   Test Case Derivation: Ensures that users can successfully submit refund requests.

Test Case FR3-T02: Retrieve Refund Requests

1. **Test ID:** FR3-T02-1

   Initial State: One or more refund requests exist in the system.

   Input: An administrator requests the list of refund requests.

   Expected Output: The system returns all refund requests along with their associated ticket and event information.

   Test Case Derivation: Confirms that administrators can review submitted refund requests.

Test Case FR3-T03: Refund Approval

1. **Test ID:** FR3-T03-1

   Initial State: A refund request exists with status *Pending*.

   Input: An administrator approves the refund request.

   Expected Output: The refund request status changes to *Approved* and the payment service processes the refund.

   Test Case Derivation: Ensures that administrators can approve refund requests and trigger the refund process.

Test Case FR3-T04: Refund Denial

1. **Test ID:** FR3-T04-1

   Initial State: A refund request exists with status *Pending*.

   Input: An administrator denies the refund request.

   Expected Output: The refund request status changes to *Denied* and the request is closed.

   Test Case Derivation: Confirms that administrators can reject refund requests and record the decision.

3.4 User Management (FR4)

The user management subsystem handles user account creation, modification, role assignment, and account deletion.

Test Case FR4-T01: Retrieve Users

1. **Test ID:** FR4-T01-1

   Initial State: User records exist in the system.

   Input: A request is made to retrieve all users.

   Expected Output: The system returns a list of users.

   Test Case Derivation: Confirms that the user controller correctly retrieves user data.

Test Case FR4-T02: Update User Information

1. **Test ID:** FR4-T02-1

   Initial State: A user account exists.

   Input: A request is made to update user information.

   Expected Output: The user record is updated and the new information is returned.

   Test Case Derivation: Ensures that user updates are processed correctly.

Test Case FR4-T03: Replace User Roles

1. **Test ID:** FR4-T03-1

   Initial State: A user has existing role assignments.

   Input: An administrator replaces the user's roles.

   Expected Output: The previous roles are removed and the new roles are assigned.

   Test Case Derivation: Confirms that role replacement operations function correctly.

3.5 Role-Based Access Control (RBAC) (FR5)

The RBAC subsystem ensures that only authorized users can access privileged operations.

The system uses a role guard to evaluate role metadata associated with protected routes and verify that the authenticated user possesses the required permissions.

Test Case FR5-T01: Access Granted for Authorized Roles

1. **Test ID:** FR5-T01-1

   Initial State: A user with the required role attempts to access a protected route.

   Input: The request passes through the role guard.

   Expected Output: Access is granted.

   Test Case Derivation: Ensures that authorized users can access protected functionality.

Test Case FR5-T02: Access Denied for Unauthorized Roles

1. **Test ID:** FR5-T02-1

   Initial State: A user without the required role attempts to access a protected route.

   Input: The request passes through the role guard.

Expected Output: The system denies access and throws an authorization error.

Test Case Derivation: Confirms that unauthorized users cannot access restricted functionality.

4 Nonfunctional Requirements Evaluation

This section evaluates whether the system satisfies the nonfunctional requirements defined in the SRS and tested according to the VnV Plan. The evaluation covers performance, usability, and security compliance. Where formal quantitative testing (e.g., timed load benchmarks, structured usability studies) was not conducted within the project timeline, the evaluation describes the alternative evidence gathered through unit tests, developer testing, and informal usability sessions with external users.

The nonfunctional areas evaluated are:

- Performance
- Usability
- Security Compliance

4.1 Performance (NFR1)

Performance was evaluated through a combination of manual developer testing during implementation and automated unit-level concurrency tests. Formal timed benchmarking with a dedicated load-testing tool (as described in the VnV Plan) was not conducted due to time constraints; however, the system's responsiveness was validated through real-device testing with external users and the concurrent checkout stress tests described in Section 6.

Test Case NFR1-T01: Registration Workflow Responsiveness

1. **Test ID:** NFR1-T01-1

   Initial State: Backend running locally with a seeded PostgreSQL database; Stripe test-mode payment integration enabled.

Input: A developer manually completes the full registration workflow on a physical mobile device: browse events, select ticket, proceed to Stripe checkout, and receive ticket confirmation.

Expected Output: The workflow completes without perceivable delay at each step. No timeouts or errors occur.

Test Case Derivation: Provides a qualitative confirmation that the registration path is responsive under single-user conditions, addressing VnV Plan NFR1-T01.

Test Case NFR1-T02: Concurrent Signup Handling

1. **Test ID:** NFR1-T02-1

   Initial State: An event with exactly one table seat remaining.

   Input: Two checkout requests are fired concurrently via `Promise.all` in a Jest unit test (test SRL6).

   Expected Output: Exactly one request succeeds; the other receives an error. No duplicate tickets or seats are assigned.

   Test Case Derivation: Validates that the `FOR UPDATE SKIP LOCKED` mechanism handles concurrent access without data corruption. While this is a unit-level simulation rather than a full HTTP load test, it directly exercises the critical concurrency code path identified in VnV Plan NFR1-T02.

Test Case NFR1-T03: Check-In Responsiveness

1. **Test ID:** NFR1-T03-1

   Initial State: Event with valid tickets issued; QR scanner screen open on a physical device.

   Input: A valid ticket QR code is scanned at the event check-in screen.

   Expected Output: The check-in confirmation appears on the admin device without perceivable delay. The backend correctly marks the ticket as checked in (verified by unit tests TP6–TP9).

   Test Case Derivation: Confirms that the check-in flow is fast enough for practical use at event doors, as required by VnV Plan NFR1-T03.

4.2 Usability (NFR2)

Usability was evaluated through informal testing sessions with peers from other capstone groups who had no prior exposure to MacSync. A formal structured usability study (as described in VnV Plan NFR3-T01) was not conducted within the project timeline; however, these sessions on physical mobile devices provided qualitative validation of the core workflows.

Test Case NFR2-T01: Registration Workflow Clarity

1. **Test ID:** NFR2-T01-1

   Initial State: MacSync deployed on a physical mobile device; peers from other capstone groups acting as first-time users.

   Input: Users complete the event browsing, signup, and ticket purchase workflow without prior instruction.

   Expected Output: Users complete the workflow without confusion or assistance. No critical navigation or comprehension issues are observed.

   Test Case Derivation: Provides qualitative evidence that the registration flow is intuitive, partially addressing VnV Plan NFR3-T01-1. External users completed the workflow without requiring guidance.

Test Case NFR2-T02: Feature Discoverability

1. **Test ID:** NFR2-T02-1

   Initial State: MacSync mobile app running with bottom tab navigation.

   Input: Users navigate to event browsing, My Tickets, and profile screens.

   Expected Output: All primary features are accessible from the bottom tab bar without requiring multi-step navigation.

   Test Case Derivation: The tab-based navigation design ensures that core features are one tap away, addressing VnV Plan NFR3-T01-2. This was confirmed during peer testing sessions.

Test Case NFR2-T03: Error Message Clarity

1. **Test ID:** NFR2-T03-1

   Initial State: User interacts with the event signup screen.

Input: User attempts to sign up for a full-capacity event or encounters a payment error.

Expected Output: A clear error message is displayed explaining the issue (e.g., "Event Full", "Payment failed"). The user is not left on a blank or broken screen.

Test Case Derivation: Error handling is verified by frontend unit tests ES7 (full event error) and backend tests SRL2–SRL3 (lockout and capacity errors), addressing VnV Plan NFR3-T01-3.

4.3 Security Compliance (NFR3)

Security was verified through automated unit tests for role-based access control and through architectural review of the payment and authentication subsystems. The RBAC guard tests (RBAC1–RBAC5) provide direct evidence that access control is enforced correctly.

Test Case NFR3-T01: Role-Based Access Control

1. **Test ID:** NFR3-T01-1

   Initial State: Users with student and admin roles exist in the test environment.

   Input: Admin users access the Create Event screen; non-admin users attempt the same. Users request their own tickets and other users' tickets.

   Expected Output: Admin-only features are hidden from non-admin users. Users can access their own tickets but not other users' tickets (unless admin). Unauthorized requests are rejected.

   Test Case Derivation: Directly tested by automated unit tests RBAC1–RBAC5. Validates VnV Plan NFR4-T01.

Test Case NFR3-T02: Sensitive Data Protection

1. **Test ID:** NFR3-T02-1

   Initial State: System uses Stripe for all payment processing; user authentication handled via Clerk.

Input: Review of the database schema and API responses for exposure of sensitive data.

Expected Output: No credit card numbers, CVVs, or payment tokens are stored in the application database. Payment data is handled entirely by Stripe. User passwords are managed by Clerk and never stored by the application.

Test Case Derivation: Architectural review confirms that the system delegates sensitive data handling to third-party services (Stripe, Clerk), satisfying VnV Plan NFR4-T03. The application database stores only Stripe session IDs and payment intent IDs for reference.

5 Comparison to Existing Implementation

This section will not be appropriate for every project.

6 Unit Testing

6.1 Event Creation and Management Module

Create Event Screen

Test ID	Inputs	Expected Values	Actual Values	Result
ECM1	Open the create event screen.	The screen displays the event form, input fields, toggles, and Save/Cancel buttons.	The screen displayed all required form controls correctly.	Pass
ECM2	Press Save with required fields left empty.	Event creation is blocked and an error message is shown for missing required fields.	Submission was blocked and an error alert was shown.	Pass
ECM3	Enter valid event data and press Save.	The event is created successfully, price is converted to cents, and the user is redirected to the confirmation page.	The event was created with the correct payload and navigation occurred.	Pass
ECM4	Enter valid data but simulate API failure on Save.	Event creation fails gracefully and an error alert is shown.	The API error was caught and the correct alert was displayed.	Pass
ECM5	Press Cancel on the create event screen.	The user is returned to the previous screen without creating an event.	The screen navigated back successfully.	Pass

Events Controller / Service

Test ID	Inputs	Expected Values	Actual Values	Result
ECM6	Request all events from the backend.	The system returns the list of stored events.	All events were returned correctly.	Pass
ECM7	Request one event by id.	The correct event is returned if it exists; otherwise null is returned.	Existing events were returned correctly and missing events returned null.	Pass
ECM8	Create an event requiring bus or table signup.	The event is created and corresponding seat records are initialized automatically.	Event creation and seat generation both succeeded.	Pass
ECM9	Update or delete an existing event.	The requested event is modified or deleted successfully.	Update and delete operations completed correctly.	Pass

6.2 Ticket Purchase and Payment System Module

Events Screen / Ticket Access

Test ID	Inputs	Expected Values	Actual Values	Result
TP1	Load an available event with price and registration count.	The event card displays the correct event name, location, price, and open status.	The event card showed the correct information.	Pass
TP2	Load an event that has reached capacity.	The event is marked Full and signup is disabled.	Full badge and disabled action were shown correctly.	Pass
TP3	Load a past event.	The event is marked Past and signup is disabled.	Past badge and ended status were shown correctly.	Pass
TP4	Click Sign Up on an available event.	The user is routed to the event signup page for that event.	Navigation to the signup page occurred successfully.	Pass
TP5	Load events for a user who is already registered.	The system shows the registered status for the user's events.	Registered events were correctly marked as signed up.	Pass

Ticket Validation / Check-In

Test ID	Inputs	Expected Values	Actual Values	Result
TP6	Submit empty or invalid QR code data.	The system rejects the QR code and returns an invalid QR error.	Invalid QR codes were rejected correctly.	Pass
TP7	Submit a valid QR code for a ticket that does not exist.	The system returns a ticket not found error.	Missing tickets were handled correctly.	Pass
TP8	Submit a valid QR code for a ticket already checked in.	The system returns an already checked-in response without updating the record.	Already checked-in tickets were detected correctly.	Pass
TP9	Submit a valid QR code for a valid unchecked ticket.	The ticket is marked as checked in and success is returned.	The ticket was updated and validated successfully.	Pass

6.3 Bus and Table Registration Module

Event Signup / Seat Initialization

Test ID	Inputs	Expected Values	Actual Values	Result
BTR1	Create an event with table signup enabled and a valid table configuration.	Table seat records are automatically created for the event.	Table seats were generated correctly.	Pass
BTR2	Create an event with bus signup enabled and a valid bus configuration.	Bus seat records are automatically created for the event.	Bus seats were generated correctly.	Pass
BTR3	Sign up a user for an event with an optional table selection.	The selected table information is passed correctly to the backend during signup.	The signup call included the correct event, user, and table id.	Pass
BTR4	Cancel an existing event signup.	The user's signup is removed successfully.	The signup was cancelled correctly.	Pass
BTR5	Attempt to cancel signup when no signup exists.	The system returns an appropriate error and does not modify data.	The system returned the expected error response.	Pass

6.4 Role-Based Access Control (RBAC) Module

Frontend and Backend Authorization

Test ID	Inputs	Expected Values	Actual Values	Result
RBAC1	Load the events screen as an admin user.	The Create Event button is visible and can be used.	The admin user could see and access event creation.	Pass
RBAC2	Load the events screen as a non-admin user.	The Create Event button is hidden.	The button was not shown to non-admin users.	Pass
RBAC3	Request a user's own tickets.	Access is allowed and the ticket list is returned.	Users could access their own tickets correctly.	Pass
RBAC4	Request another user's tickets as an admin.	Access is allowed and the ticket list is returned.	Admin access was granted correctly.	Pass
RBAC5	Request another user's tickets as a non-admin.	Access is denied and an error is returned.	Unauthorized access was rejected correctly.	Pass

6.5 Event Signup Screen Module

Event Signup Page (Frontend)

These tests verify the `EventSignupScreen` React Native component that handles the full event registration workflow, including displaying event details, processing free and paid signups, and table selection.

Test ID	Inputs	Expected Values	Actual Values	Result
ES1	Load the event signup page for an event with all fields populated.	The page displays the event title, date, location, price, description, and an Open status badge.	All event details and the Open badge were displayed correctly.	Pass
ES2	Load the signup page for a full-capacity event and a past event.	Full badge shown when at capacity; Past badge and "Event Ended" text for past events.	Status badges rendered correctly for both scenarios.	Pass
ES3	Press "Back to Events" or "Cancel" on the signup page.	The router navigates back to the events list.	Navigation occurred correctly in both cases.	Pass
ES4	Confirm signup on a free event.	The `signupForEvent` API is called and a success notification is displayed.	API was called with correct parameters and success was shown.	Pass
ES5	Confirm signup on a paid event.	The `createCheckoutSession` API is called with the correct success and cancel URLs.	Checkout session was created with the correct redirect URLs.	Pass
ES6	Load the signup page for an event with table selection required (`tableCount = 3`).	A Table Selection card is visible and the picker lists Table 1 through Table 3.	Table picker appeared and listed all available tables.	Pass
ES7	Attempt to confirm signup on a full-capacity event.	"Event Full" button is shown; clicking it displays an error notification.	Full state and error notification were displayed correctly.	Pass

My Tickets Screen (Frontend)

This test verifies the `MyTickets` component that displays the user's registered event tickets.

Test ID	Inputs	Expected Values	Actual Values	Result
MT1	Load the My Tickets screen for a user with existing tickets.	Each ticket card displays the event name, date, location, and price.	Ticket details were rendered correctly for all tickets.	Pass

6.6 Seat Reservation Lockout Module

These tests verify the concurrency-safe seat reservation mechanism used during paid event checkout. The system uses PostgreSQL `FOR UPDATE SKIP LOCKED` within a database transaction to atomically lock a seat row, preventing two concurrent users from both entering the payment flow for the same seat. A `seat_reservations` table holds the lock while the user completes Stripe checkout.

Checkout Session and Reservation (Backend)

Test ID	Inputs	Expected Values	Actual Values	Result
SRL1	A single user requests a checkout session for an event with available table seats.	A Stripe checkout URL and session ID are returned; a seat reservation record is created.	Checkout session was created and reservation was inserted successfully.	Pass
SRL2	A user requests a checkout session when all table seats are locked by other users.	The system returns an error containing "no table seats left".	The lockout error was returned correctly with no checkout URL.	Pass
SRL3	A user requests a checkout session when the event has reached full capacity.	The system returns "Event is full" before entering the transaction.	The capacity check prevented the checkout session creation.	Pass
SRL4	A checkout session is cancelled or expires, providing the Stripe session ID.	The reservation row is deleted from the database and the seat becomes available.	The reservation was released and the delete operation completed.	Pass
SRL5	A payment completes successfully for a valid reservation with a table seat.	A ticket is created, the seat is assigned, payment is recorded, and the reservation is deleted.	The full completion flow succeeded and all database operations were verified.	Pass
SRL6	Two users fire checkout requests concurrently for an event with exactly one seat remaining.	Exactly one user succeeds with a checkout URL; the other receives "no table seats left".	One request succeeded and one failed as expected; the race condition was handled correctly.	Pass

6.7 Waivers and Preference Submission Module

At the time of testing, no dedicated automated unit tests were implemented for waiver submission or preference submission as separate isolated modules. These behaviours were instead covered indirectly through broader event registration and ticketing workflows. Additional direct tests for waiver completion validation and user preference persistence may be added in future revisions.

7 Changes Due to Testing

This section documents how feedback from the Rev 0 demo, supervisor review, and subsequent testing activities shaped the final product.

7.1 Refund Functionality

The Rev 0 feedback noted that *"request refund not yet working."* At the time of the demo, the "Request Refund" button on the My Tickets screen displayed a "coming soon" placeholder. The backend `PaymentsService.refundPayment` method and `POST /payments/:paymentId/refund` endpoint have since been implemented using the Stripe Refunds API. The frontend button now triggers the refund workflow for paid tickets, while free-event tickets use the direct cancel flow.

7.2 Admin Event Editing

The Rev 0 feedback noted that *"don't currently have the ability for the administrators to edit a created event."* The admin web dashboard event detail page was updated from a placeholder to a fully functional event editing form, allowing administrators to modify event name, date, location, capacity, price, and signup options.

8 Automated Testing

Automated testing forms the foundation of the verification process for the platform, as outlined in the VnV Plan. The current implementation uses

Jest with **ts-jest** for both frontend and backend, executed via GitHub Actions on each pull request.

Backend tests (run with `npm run test:backend` from the `test` directory) exercise NestJS controllers and services in isolation. The suite includes unit tests for the users controller and service (including RBAC and role management), events controller and service, refund requests, payments, and the roles guard. External dependencies such as Drizzle ORM and Stripe are mocked to ensure deterministic, fast execution. Coverage is configured for the events, auth, and users modules. Tests are written in TypeScript with Jest mocks and a chainable database helper to simulate query results.

Frontend tests (run with `npm run test:frontend`) target the Expo React Native mobile app using a jsdom environment. The React Testing Library is used to verify components and workflows such as event creation, the events screen, ticket details, refund requests, and QR scanning. React Native, Expo Router, and other native modules are mocked so tests run in Node without a device or simulator. Coverage targets the `src/frontend/mobile/app` directory.

Continuous integration is implemented with GitHub Actions. The CI workflow runs on pull requests and includes: backend lint and build (with a PostgreSQL service container for migrations), backend unit tests, frontend unit tests, web-admin lint and build, and mobile type-checking and Expo config validation. Test dependencies are installed from the dedicated `test` package, which also defines `npm run coverage:frontend` and `coverage:backend` for local coverage reporting.

This approach aligns with the VnV Plan's emphasis on Jest for the backend, automated CI execution, and separation of frontend and backend test suites. The frontend uses Jest rather than the originally planned Vitest; Cypress end-to-end tests and Supertest-based API integration tests are not yet in place. Static analysis (ESLint, Prettier, TypeScript) is applied in the backend and web-admin CI jobs, consistent with the plan's toolchain requirements.

9 Trace to Requirements

Table 1 maps the selected tests to the main functional requirements. A checkmark indicates that the test provides evidence for the corresponding requirement.

Table 1: Traceability Between Tests and Functional Requirements

Test ID	FR1	FR2	FR3	FR4	FR5
ECM2					✓
ECM3					✓
ECM4					✓
ECM8		✓			✓
ECM9					✓
TP1	✓				
TP2	✓				
TP4	✓				
TP5	✓				
TP6	✓				
TP9	✓				
BTR1		✓			✓
BTR2		✓			✓
BTR3		✓			
BTR4		✓			
BTR5		✓			
RBAC1				✓	✓
RBAC2				✓	
RBAC3	✓			✓	
RBAC4	✓			✓	
RBAC5				✓	
FS1	✓				

10 Trace to Modules

Table 2 maps the selected tests to the major functional modules of the system. A checkmark indicates that the test belongs to or primarily validates that module.

Table 2: Traceability Between Tests and Modules

Test ID	TPPS	BTR	WPS	RBAC	ECM
ECM2					✓
ECM3					✓
ECM4					✓
ECM8		✓			✓
ECM9					✓
TP1	✓				
TP2	✓				
TP4	✓				
TP5	✓				
TP6	✓				
TP9	✓				
BTR1		✓			✓
BTR2		✓			✓
BTR3		✓			
BTR4		✓			
BTR5		✓			
RBAC1				✓	✓
RBAC2				✓	
RBAC3	✓			✓	
RBAC4	✓			✓	
RBAC5				✓	
ES1	✓				
ES4	✓				
ES5	✓				
ES6		✓			

11 Code Coverage Metrics

The VnV Plan specifies that the CI pipeline produce code coverage reports summarizing the percentage of tested lines and functions, to be reviewed at the end of each sprint, with a goal of 75% across all core modules. The current implementation supports coverage collection via Jest's built-in instrumentation.

Backend coverage is configured in `test/backend/jest.config.js`. The `collectCoverageFrom` setting includes:

- `src/backend/src/events/**/*.ts` (excluding `*.d.ts` and `*.module.ts`)
- `src/backend/src/auth/**/*.ts`
- `src/backend/src/users/**/*.ts`

Coverage reports are written to `test/coverage/backend`. The events, auth, and users modules correspond to the core backend logic for event management, RBAC, and user administration tested by the unit tests described in Section 6.

Frontend coverage is configured in `test/frontend/jest.config.js`. Coverage is collected from `src/frontend/mobile/app/**/*.{ts,tsx}`, excluding type definitions and `_layout.tsx`. Reports are written to `test/coverage/frontend`. This scope aligns the mobile app screens (event creation, events list, ticket detail, refund requests, QR scanner) with the frontend tests.

Coverage is generated locally by running `npm run coverage:backend` and `npm run coverage:frontend` (or `npm run coverage` for both) from the `test` directory. The CI workflow currently runs the test suites without the `--coverage` flag; coverage reporting in CI and enforcement of the 75% target can be added in a future iteration. The VnV Plan also mentions mutation testing to quantify the effectiveness of the test plan; mutation testing is not yet integrated into the build or CI pipeline.

References

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Reflection.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?

4. In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV? If there were differences, what changes required the modification in the plan? Why did these changes occur? Would you be able to anticipate these changes in future projects? If there weren't any differences, how was your team able to clearly predict a feasible amount of effort and the right tasks needed to build the evidence that demonstrates the required quality? (It is expected that most teams will have had to deviate from their original VnV Plan.)